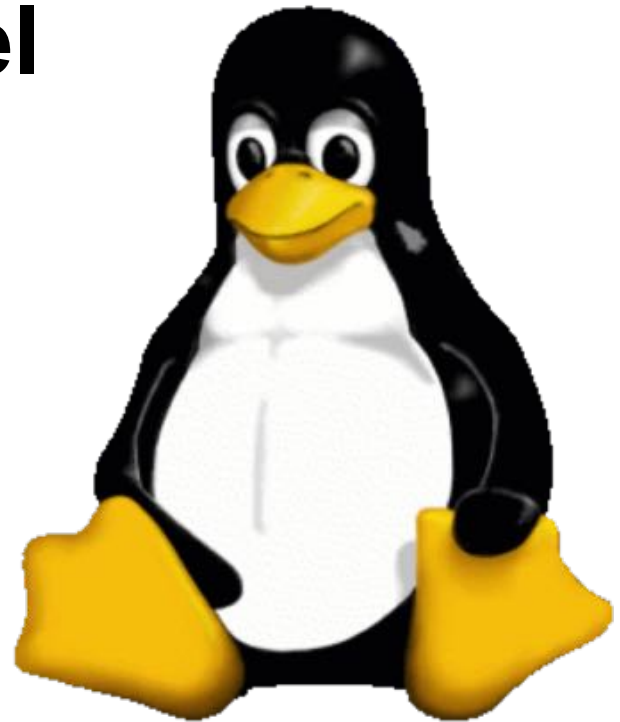


CS353 Linux Kernel

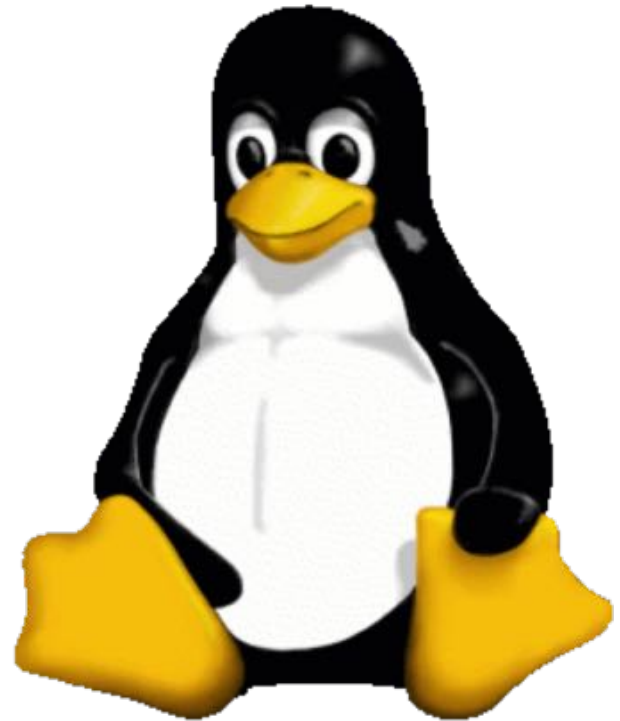
Quan Chen
Dept. of CSE, SJTU
chen-quan@cs.sjtu.edu.cn



上海交通大學

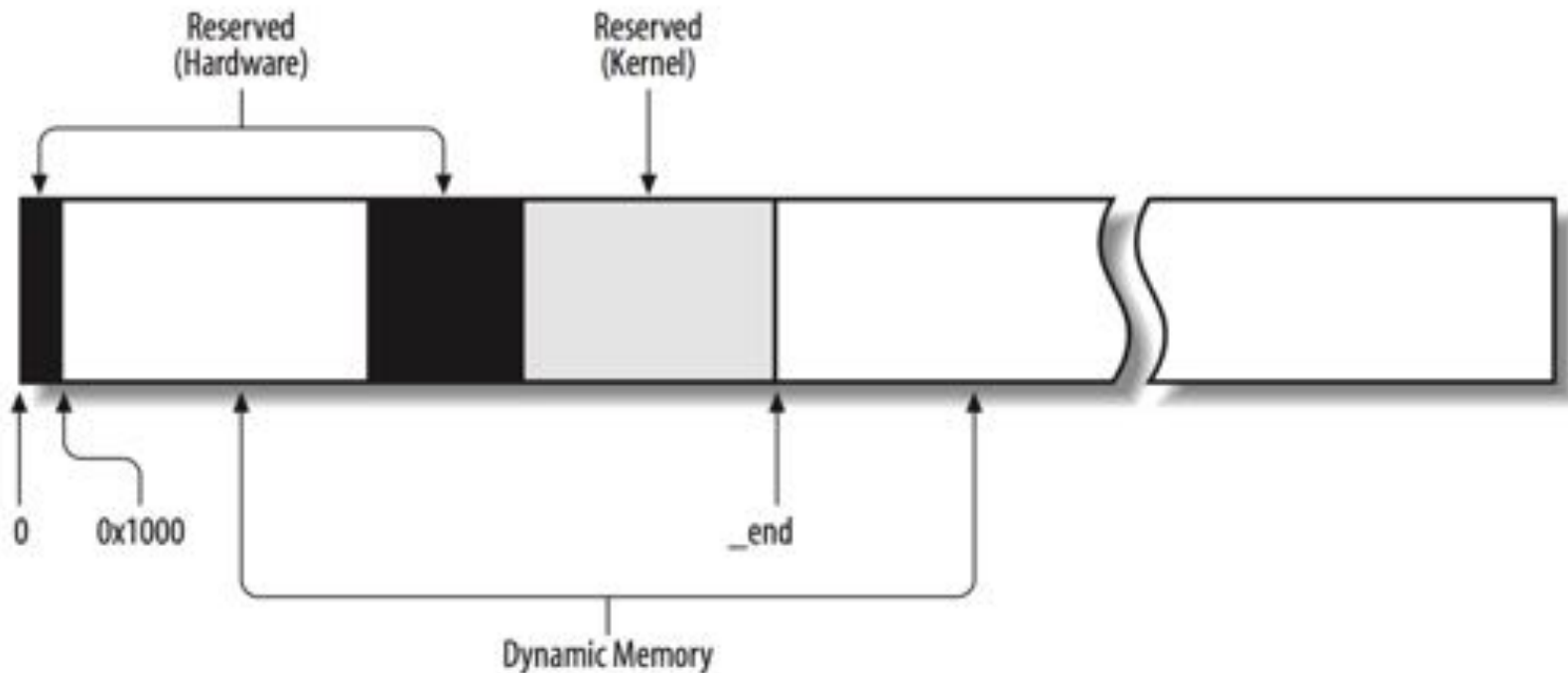
6B. Memory Management -- Methods

Quan Chen
Dept. of CSE, SJTU
chen-quan@cs.sjtu.edu.cn



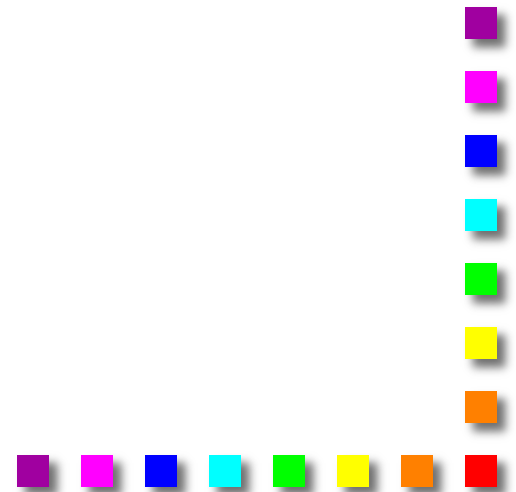
上海交通大學

Page Frames in Dynamic Memory



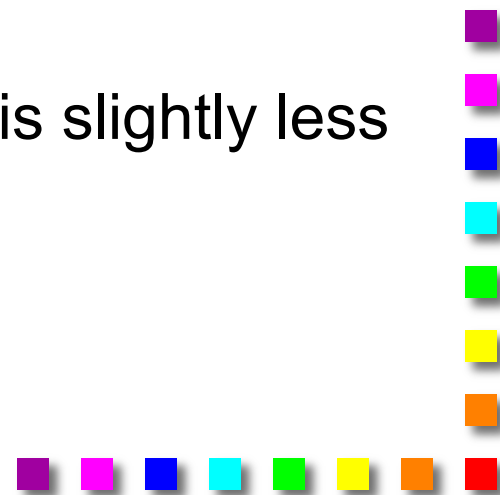
Outline

- Page Frame Management
- Memory Area Management
- Noncontiguous Memory Area Management



Page Frame Management

- Two different page frame sizes
 - 4KB: standard memory allocation unit
 - 4MB
- Page Descriptors
 - Of type *page* (32 bytes)
 - Stored in the *mem_map* array
 - Space required by *mem_map* array is slightly less than 1% of the whole RAM (why?)



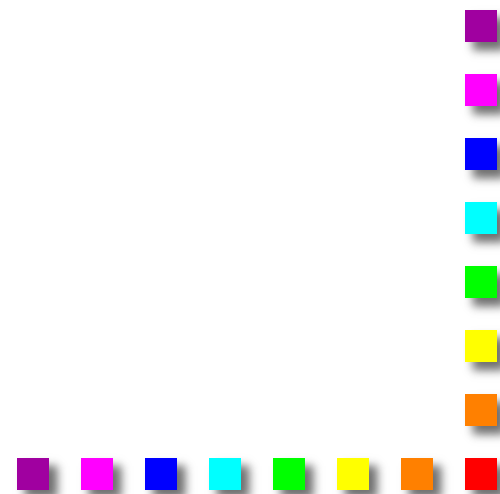
The Fields of the Page Descriptor

Type	Name	Description
unsigned long	flags	Array of flags
atomic_t	_count	Page frame's reference counter
atomic_t	_mapcount	Number of Page Table entries that refer to the page frame
unsigned long	private	Available to the kernel component that is using the page
struct address_space *	mapping	Used when the page is inserted into the page cache
unsigned long	index	Used by several kernel components with different meanings
Struct list_head	lru	Contains pointers to the least recently used doubly linked list of pages.



Flags Describing the Status of a Page Frame

- `PG_locked`, `PG_error`, `PG_referenced`, `PG_uptodate`, `PG_dirty`, `PG_lru`, `PG_active`, `PG_slab`, `PG_skip`, `PG_highmem`, `PG_checked`, `PG_arch_1`, `PG_reserved`, `PG_private`, `PG_writeback`, `PG_nosave`, `PG_compound`, `PG_swapcache`, `PG_mappedtodisk`, `PG_reclaim`, `PG_nosave_free`



NUMA (Non-Uniform Memory Access)

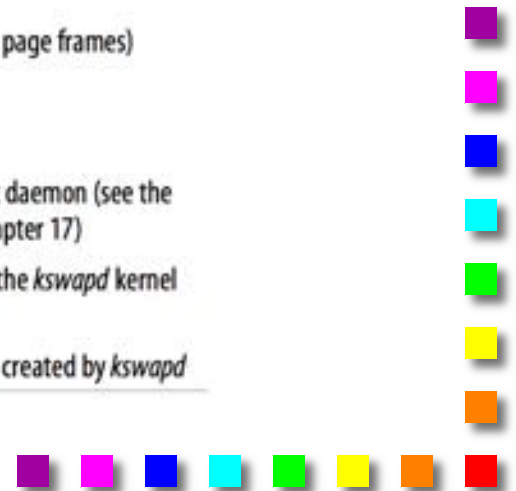
- The physical memory is partitioned in several nodes
 - Each node has a descriptor (Table 8-3)
 - The physical memory inside each node can be split into several zones
 - ZONE_DMA: < 16MB
 - ZONE_NORMAL: 16MB-896MB
 - ZONE_HIGHMEM: > 896MB
 - Each zone has its descriptor (Table 8-4)



Node Descriptor

- Each node has a descriptor of type *pg_data_t*

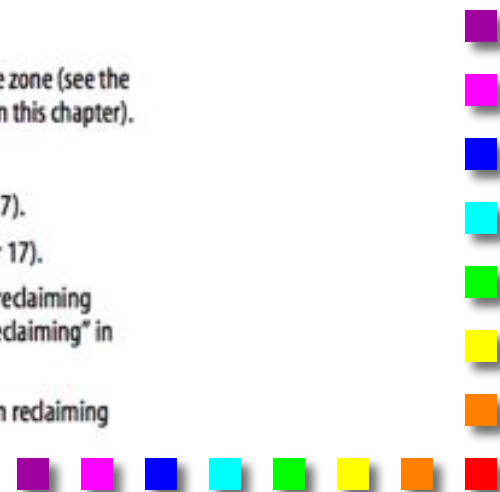
Type	Name	Description
struct zone []	node_zones	Array of zone descriptors of the node
struct zonelist []	node_zonelists	Array of zonelist data structures used by the page allocator (see the later section "Memory Zones")
int	nr_zones	Number of zones in the node
struct page *	node_mem_map	Array of page descriptors of the node
struct bootmem_data *	bdata	Used in the kernel initialization phase
unsigned long	node_start_pfn	Index of the first page frame in the node
unsigned long	node_present_pages	Size of the memory node, excluding holes (in page frames)
unsigned long	node_spanned_pages	Size of the node, including holes (in page frames)
int	node_id	Identifier of the node
pg_data_t *	pgdat_next	Next item in the memory node list
wait_queue_head_t	kswapd_wait	Wait queue for the <i>kswapd</i> pageout daemon (see the section "Periodic Reclaiming" in Chapter 17)
struct task_struct *	kswapd	Pointer to the process descriptor of the <i>kswapd</i> kernel thread
int	kswapd_max_order	Logarithmic size of free blocks to be created by <i>kswapd</i>



Zone Descriptor

- Each zone has a descriptor of type *zone*

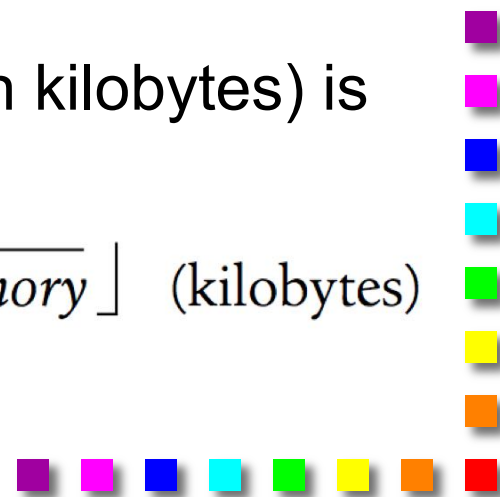
Type	Name	Description
unsigned long	free_pages	Number of free pages in the zone.
unsigned long	pages_min	Number of reserved pages of the zone (see the section “The Pool of Reserved Page Frames” later in this chapter).
unsigned long	pages_low	Low watermark for page frame reclaiming; also used by the zone allocator as a threshold value (see the section “The Zone Allocator” later in this chapter).
unsigned long	pages_high	High watermark for page frame reclaiming; also used by the zone allocator as a threshold value.
unsigned long []	lowmem_reserve	Specifies how many page frames in each zone must be reserved for handling low-on-memory critical situations.
struct per_cpu_pageset[]	pageset	Data structure used to implement special caches of single page frames (see the section “The Per-CPU Page Frame Cache” later in this chapter).
spinlock_t	lock	Spin lock protecting the descriptor.
struct free_area []	free_area	Identifies the blocks of free page frames in the zone (see the section “The Buddy System Algorithm” later in this chapter).
spinlock_t	lru_lock	Spin lock for the active and inactive lists.
struct list head	active_list	List of active pages in the zone (see Chapter 17).
struct list head	inactive_list	List of inactive pages in the zone (see Chapter 17).
unsigned long	nr_scan_active	Number of active pages to be scanned when reclaiming memory (see the section “Low On Memory Reclaiming” in Chapter 17).
unsigned long	nr_scan_inactive	Number of inactive pages to be scanned when reclaiming memory.



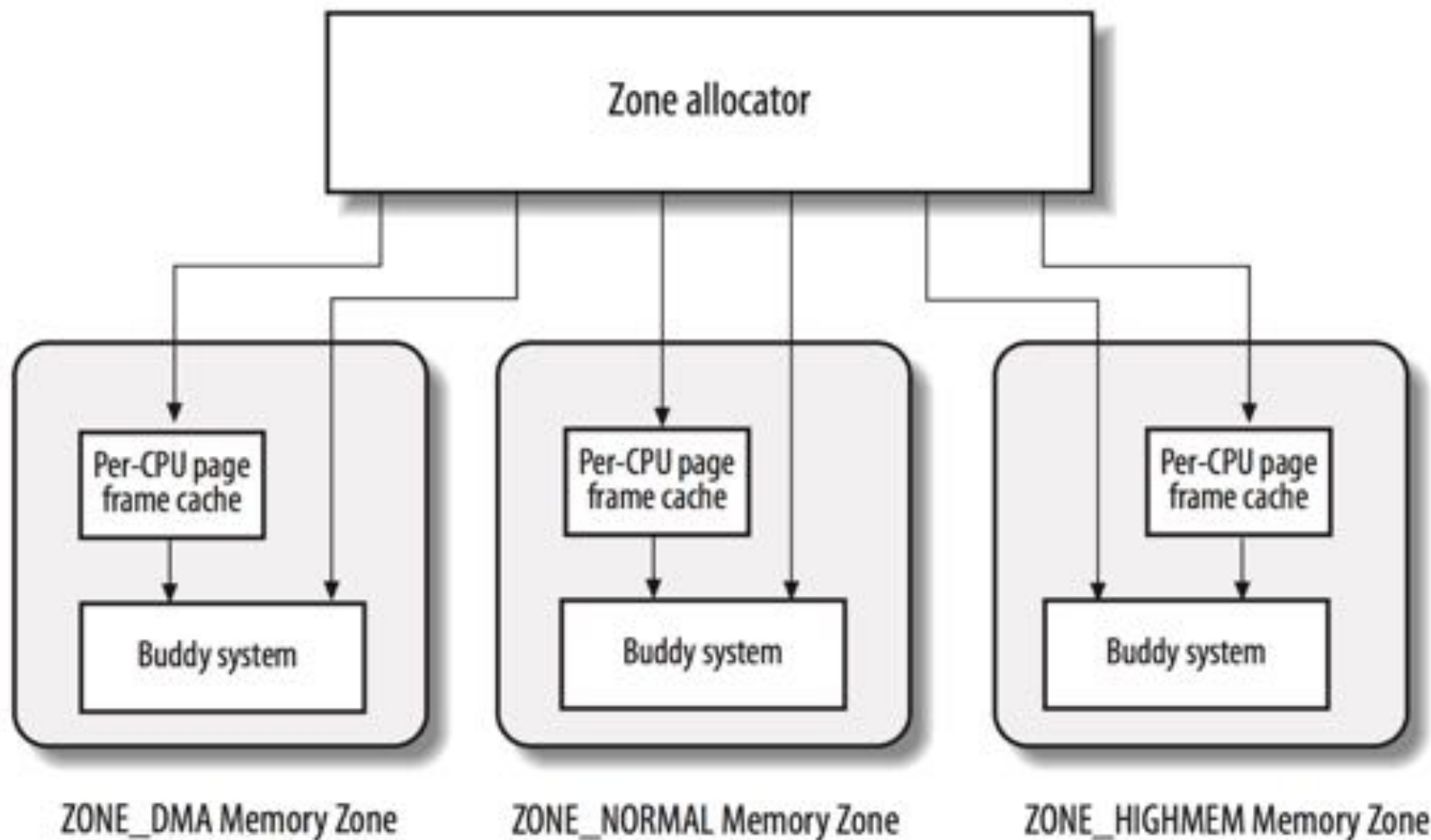
The Pool of Reserved Page Frames

- Some kernel control paths cannot be blocked while requesting memory (**Atomic allocation request**)
- Kernel reserves a pool of page frames for atomic memory allocation requests
- The amount of the reserved memory (in kilobytes) is stored in the *min_free_kbytes* variable

$$\text{reserved pool size} = \lfloor \sqrt{16 \times \text{directly mapped memory}} \rfloor \quad (\text{kilobytes})$$



Zoned Page Frame Allocator



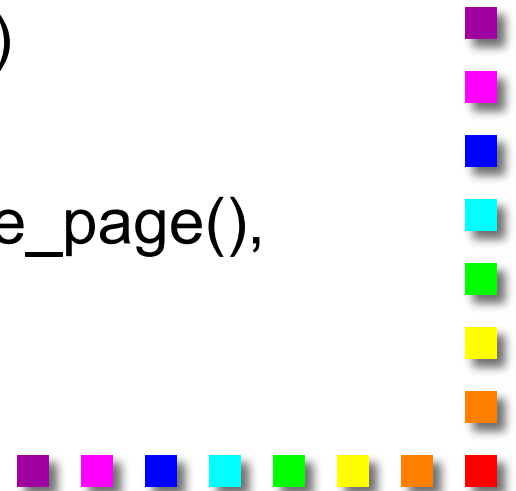
Request and Release Page Frames

■ Request:

- `alloc_pages(gfp_mask, order),`
`alloc_page(gfp_mask),`
`__get_free_pages(gfp_mask, order),`
`__get_free_page(gfp_mask),`
`get_zeroed_page(gfp_mask),`
`__get_dma_pages(gfp_mask, order)`

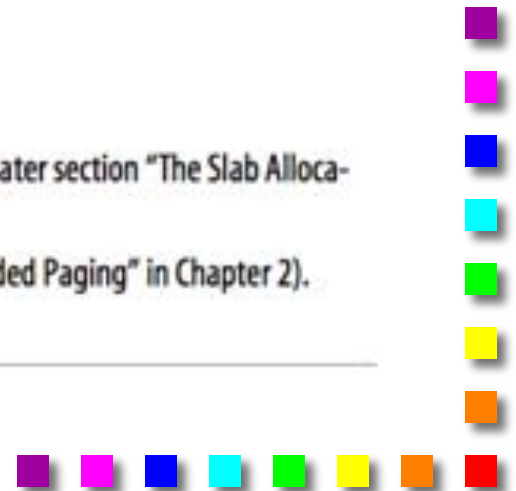
■ Release:

- `__free_pages(), free_pages(), __free_page(),`
`free_page(),`



Flags Used to Request Page Frames

Flag	Description
__GFP_DMA	The page frame must belong to the ZONE_DMA memory zone. Equivalent to GFP_DMA.
__GFP_HIGHMEM	The page frame may belong to the ZONE_HIGHMEM memory zone.
__GFP_WAIT	The kernel is allowed to block the current process waiting for free page frames.
__GFP_HIGH	The kernel is allowed to access the pool of reserved page frames.
__GFP_IO	The kernel is allowed to perform I/O transfers on low memory pages in order to free page frames.
__GFP_FS	If clear, the kernel is not allowed to perform filesystem-dependent operations.
__GFP_COLD	The requested page frames may be "cold" (see the later section "The Per-CPU Page Frame Cache").
__GFP_NOWARN	A memory allocation failure will not produce a warning message.
__GFP_REPEAT	The kernel keeps retrying the memory allocation until it succeeds.
__GFP_NOFAIL	Same as __GFP_REPEAT.
__GFP_NORETRY	Do not retry a failed memory allocation.
__GFP_NO_GROW	The slab allocator does not allow a slab cache to be enlarged (see the later section "The Slab Allocator").
__GFP_COMP	The page frame belongs to an extended page (see the section "Extended Paging" in Chapter 2).
__GFP_ZERO	The page frame returned, if any, must be filled with zeros.



Buddy System Algorithm (1)

- Two methods of avoiding external fragmentation
 - Use the paging unit to map groups of noncontiguous free page frames into intervals of contiguous linear addresses.
 - Avoiding as much as possible the need to split up a large free block to satisfy a request for a smaller one



Buddy System Algorithm (2)

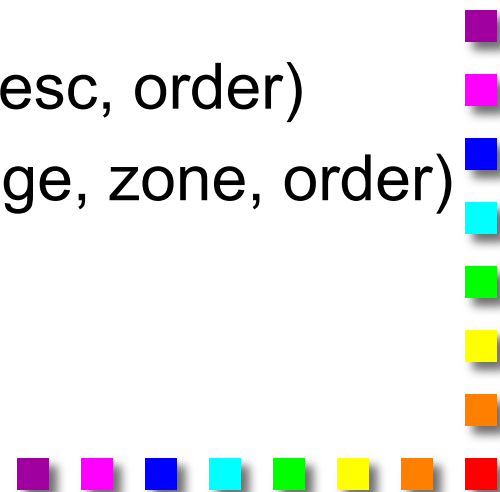
- To avoid **external fragmentation** without paging
 - Contiguous page frames are sometimes necessary
 - Advantage of leaving kernel page tables unchanged
 - Large chunks of contiguous physical memory can be accessed by the kernel through 4MB pages



Buddy System Algorithm (3)

- The Buddy System

- 11 lists of blocks: groups of 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024 contiguous page frames
- 3 buddy systems in Linux 2.6
 - For DMA, normal page frames, and high-memory page frames
- Allocate a block: `__rmqueue(zone desc, order)`
- Free a block: `__free_pages_bulk(page, zone, order)`



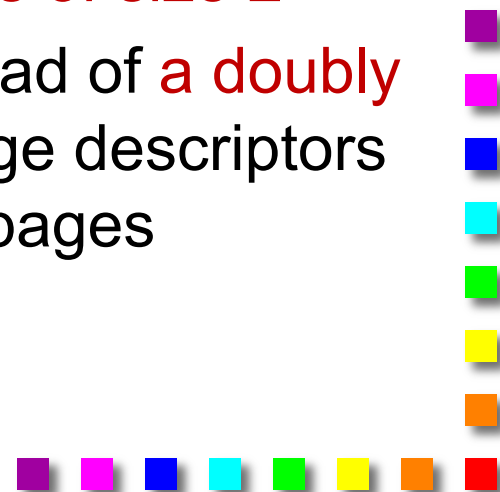
When two blocks are buddies?

- Both blocks have the same size, b .
- They are located in contiguous physical addresses.
- The physical address of the first page frame of the first block is a multiple of $2 \times b \times 2^{12}$.



Data Structures of Buddy System

- An array consisting of **eleven** elements of type *free_area*, one element for each group size. The array is stored in the *free_area* field of the zone descriptor.
- the ***k*th element** of the *free_area* array in the zone descriptor, identifies **all the free blocks of size 2^k**
- *free_list* field of this element is the head of **a doubly linked circular list** that collects the page descriptors associated with the free blocks of 2^k pages



Per-CPU Page Frame Caches

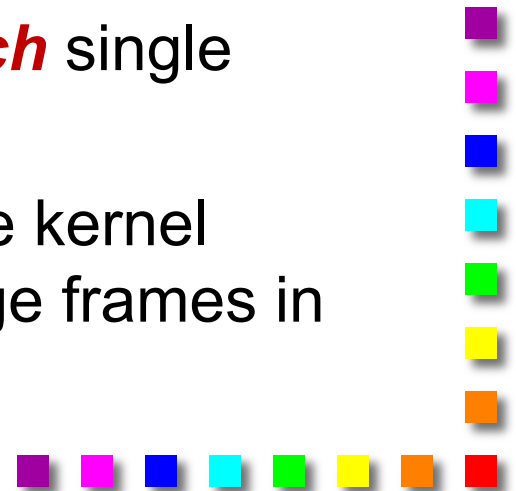
- Each per-CPU cache includes some pre-allocated page frames for single memory requests
 - Hot cache : page frames whose contents are likely to be included in hardware cache
 - Cold cache
- The fields of *per_cpu_pages* descriptor (Table 8-7)
- Allocating page frames: buffered_rmqueue()
- Releasing page frames: free_hot_page(), free_cold_page()



Per_cpu_pages descriptor

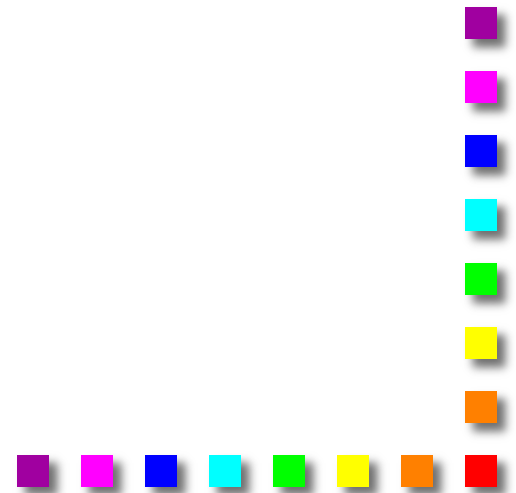
Type	Name	Description
int	count	Number of pages frame in the cache
int	low	Low watermark for cache replenishing
int	high	High watermark for cache depletion
int	batch	Number of page frames to be added or subtracted from the cache
struct list_head	list	List of descriptors of the page frames included in the cache

- If the number of page frames $< \text{low}$, the kernel replenishes the cache by allocating **batch** single page frames from the buddy system
- If the number of page frames $> \text{high}$, the kernel releases to the buddy system **batch** page frames in the cache



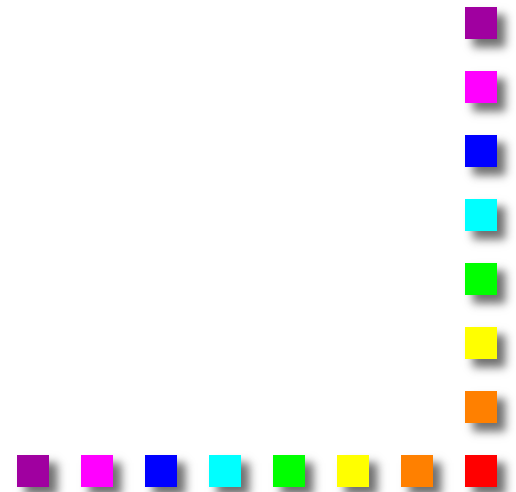
Outline

- Page Frame Management
- Memory Area Management
- Noncontiguous Memory Area Management



Memory Area Management

- To avoid internal fragmentation
 - Early Linux version adopt *buddy system*
 - Not efficient
 - A better algorithm is derived from *slab allocator*
 - Adopted in Solaris 2.4



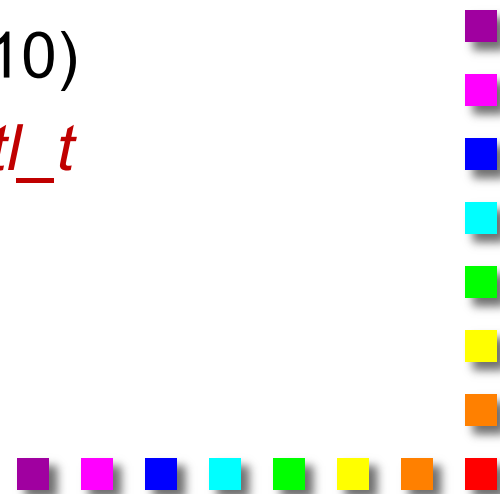
Assumption with Slab

- A slab allocator views the memory areas as *objects* consisting of both a set of data structures and a couple of functions called *constructor* and *destructor*
- The kernel functions tend to request memory areas of the same type repeatedly
- Requests for memory areas can be classified according to their frequencies
- Every call to a buddy system function “dirties” the hardware cache, thus increase the average memory access time

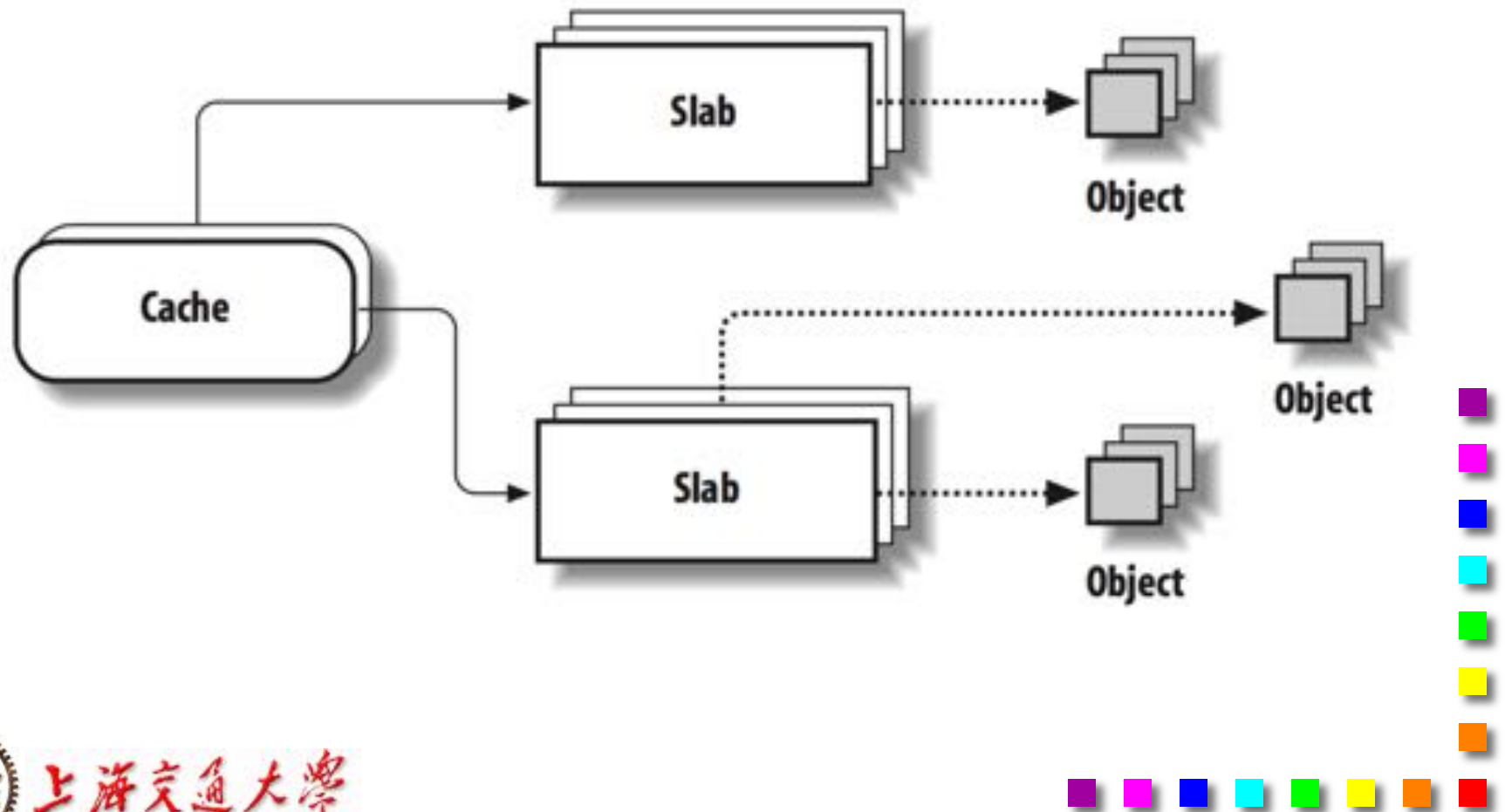


The Slab Allocator

- The slab allocator groups objects into *cache*s
 - Each cache is a store of objects of the same type
 - A *cache* is divided into *slabs*
 - Each slab consists of one or more contiguous page frames that contain both allocated and free *objects*
 - Cache descriptor of type *kmem_cache_t* (Table 8-8)
 - Slab descriptor of type *slab* (Table 8-10)
 - Object descriptor of type *kmem_bufctl_t*



The Slab Allocator Components



kmem_cache_t Descriptor

Type	Name	Description
struct array_cache * []	array	Per-CPU array of pointers to local caches of free objects (see the section "Local Caches of Free Slab Objects" later in this chapter).
unsigned int	batchcount	Number of objects to be transferred in bulk to or from the local caches.
unsigned int	limit	Maximum number of free objects in the local caches. This is tunable.
struct kmem_list3	lists	See next table.
unsigned int	objsize	Size of the objects included in the cache.
unsigned int	flags	Set of flags that describes permanent properties of the cache.
unsigned int	num	Number of objects packed into a single slab. (All slabs of the cache have the same size.)
unsigned int	free_limit	Upper limit of free objects in the whole slab cache.
spinlock_t	spinlock	Cache spin lock.
unsigned int	gfporder	Logarithm of the number of contiguous page frames included in a single slab.
unsigned int	gfpflags	Set of flags passed to the buddy system function when allocating page frames.
size_t	colour	Number of colors for the slabs (see the section "Slab Coloring" later in this chapter).

...

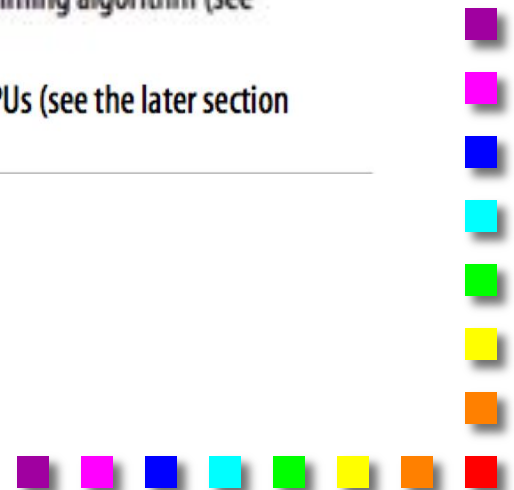


上海交通大学



kmem_list3 Structure

Type	Name	Description
struct list_head	slabs_partial	Doubly linked circular list of slab descriptors with both free and non-free objects
struct list_head	slabs_full	Doubly linked circular list of slab descriptors with no free objects
struct list_head	slabs_free	Doubly linked circular list of slab descriptors with free objects only
unsigned long	free_objects	Number of free objects in the cache
int	free_touched	Used by the slab allocator's page reclaiming algorithm (see Chapter 17)
unsigned long	next_reap	Used by the slab allocator's page reclaiming algorithm (see Chapter 17)
struct array_cache *	shared	Pointer to a local cache shared by all CPUs (see the later section "Local Caches of Free Slab Objects")

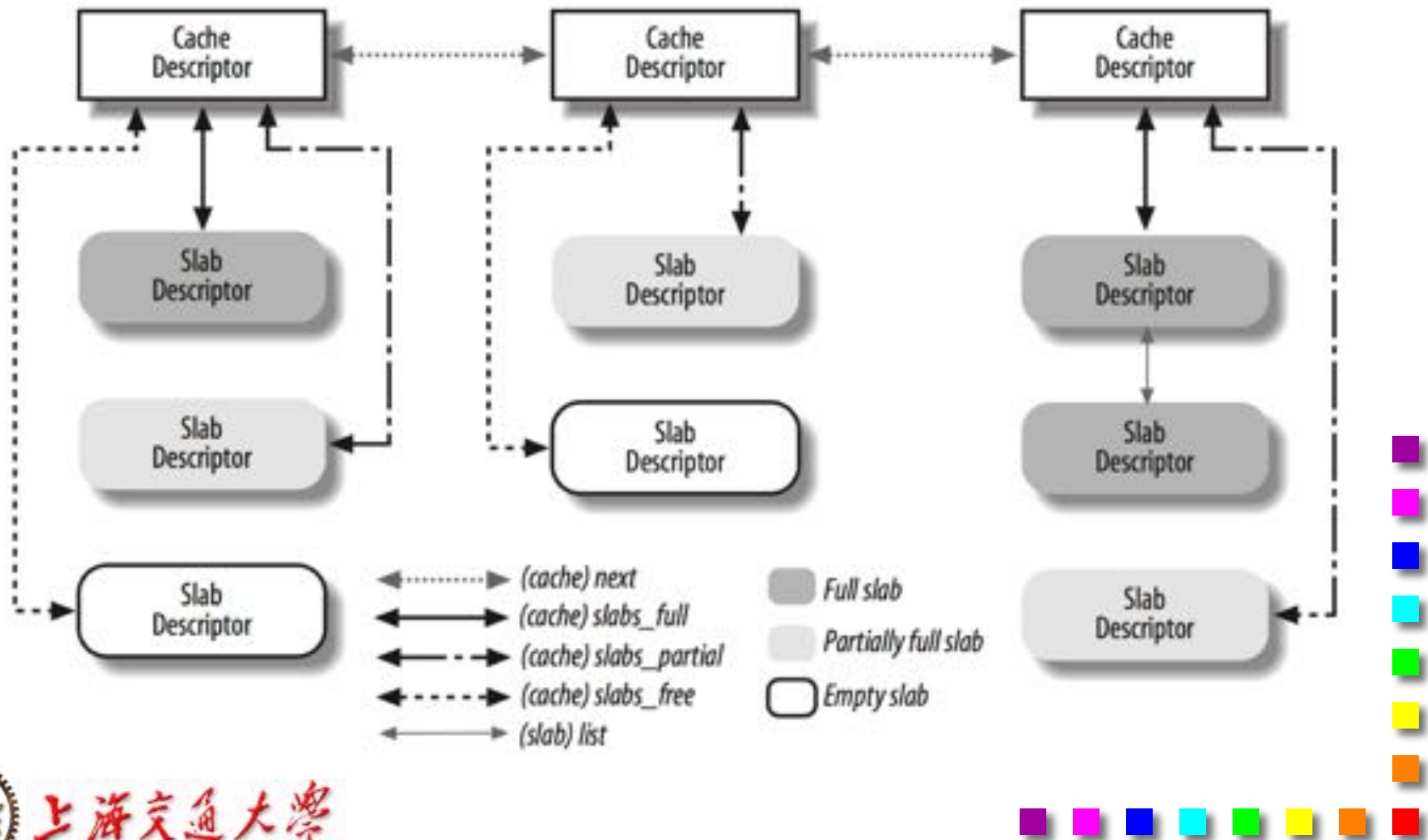


Slab Descriptor

Type	Name	Description
struct list_head	list	Pointers for one of the three doubly linked list of slab descriptors (either the slabs_full, slabs_partial, or slabs_free list in the kmem_list3 structure of the cache descriptor)
unsigned long	colouroff	Offset of the first object in the slab (see the section "Slab Coloring" later in this chapter)
void *	s_mem	Address of first object (either allocated or free) in the slab
unsigned int	inuse	Number of objects in the slab that are currently used (not free)
unsigned int	free	Index of next free object in the slab, or BUFCTL_END if there are no free objects left (see the section "Object Descriptor" later in this chapter)



Relationship between Cache and Slab Descriptors



General vs. Specific Caches (1)

- General cache: `kmem_cache_init()`
 - `kmem_cache`
 - 26 caches: two caches for each of the 13 sizes
 - A table called `malloc_sizes` (whose elements are of type `cache_sizes`) points to 26 cache descriptors associated with memory areas of size 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536, and 131072 bytes
- Specific cache: `kmem_cache_create()`



When to Allocate or Release?

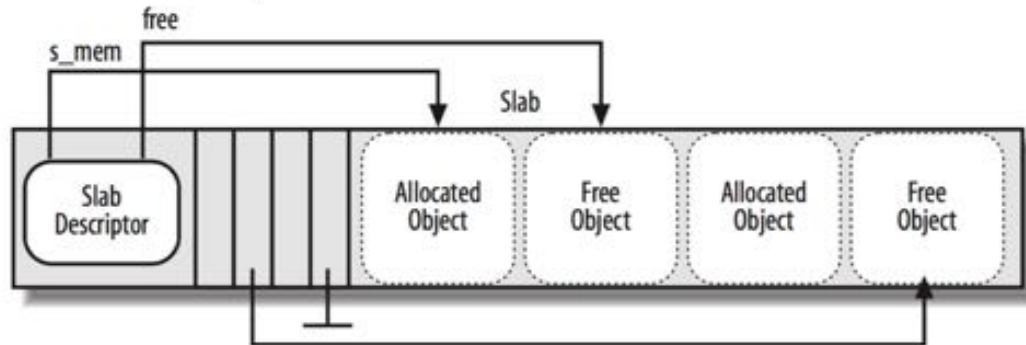
- Allocating a slab to a cache
 - When: A request has been issued to allocate a new object, and the cache does not include a free object.
 - How: `cache_grow()`
- Releasing a slab from a cache
 - When: Too many free objects in the slab cache; and a timer function periodically determines that there are fully unused slabs that can be released
 - How: `slab_destroy()`



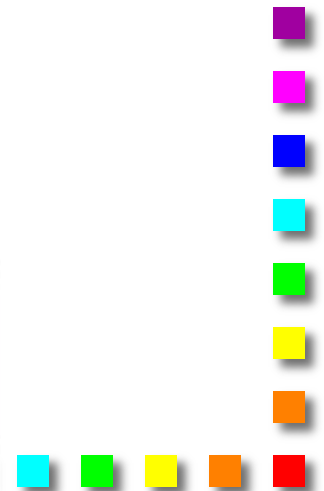
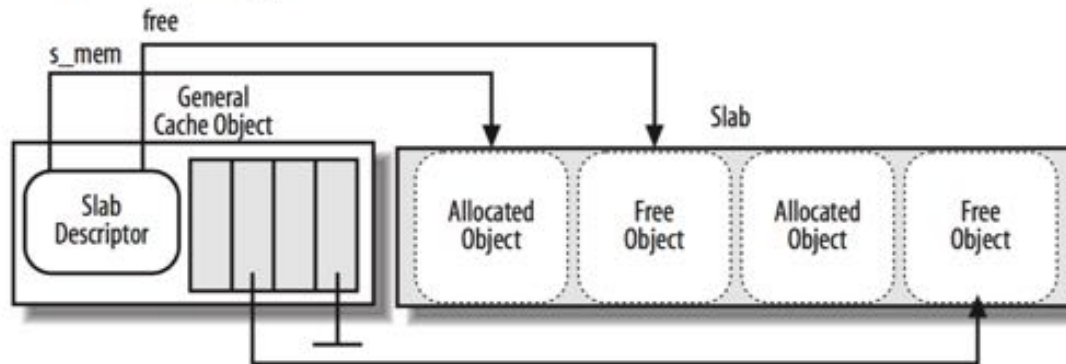
Slab and Object Descriptors (1)

- An object descriptor contains the **index of the next free object** in the slab, thus implementing a simple list of free objects inside the slab.

Slab with Internal Descriptors

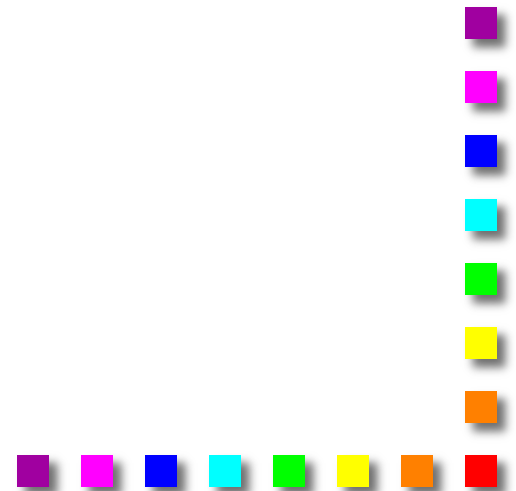


Slab with External Descriptors



Slab and Object Descriptors (2)

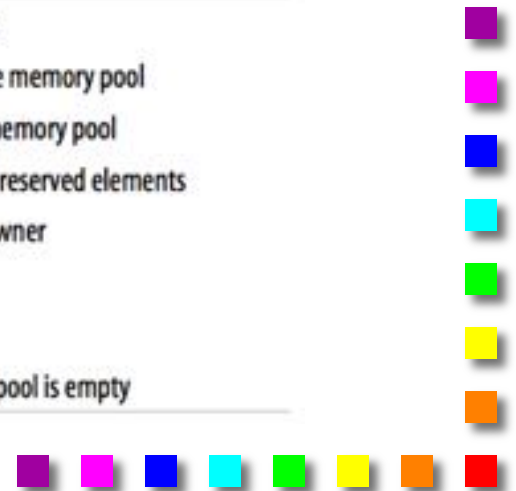
- Allocating a slab object
 - `kmem_cache_alloc()`
- Freeing a slab object
 - `kmem_cache_free()`
- General purpose objects
 - `kmalloc()`
 - `kfree()`



Memory Pools

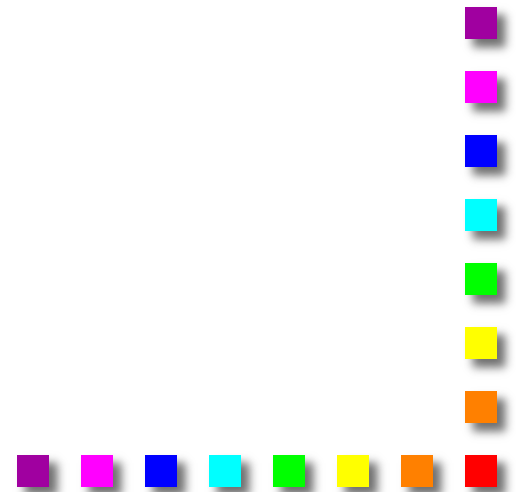
- New in Linux 2.6
- Type *mempool_t*
- `mempool_alloc()`
- `mempool_free()`

Type	Name	Description
spinlock_t	lock	Spin lock protecting the object fields
int	min_nr	Maximum number of elements in the memory pool
int	curr_nr	Current number of elements in the memory pool
void **	elements	Pointer to an array of pointers to the reserved elements
void *	pool_data	Private data available to the pool's owner
mempool_alloc_t *	alloc	Method to allocate an element
mempool_free_t *	free	Method to free an element
wait_queue_head_t	wait	Wait queue used when the memory pool is empty



Outline

- Page Frame Management
- Memory Area Management
- Noncontiguous Memory Area Management



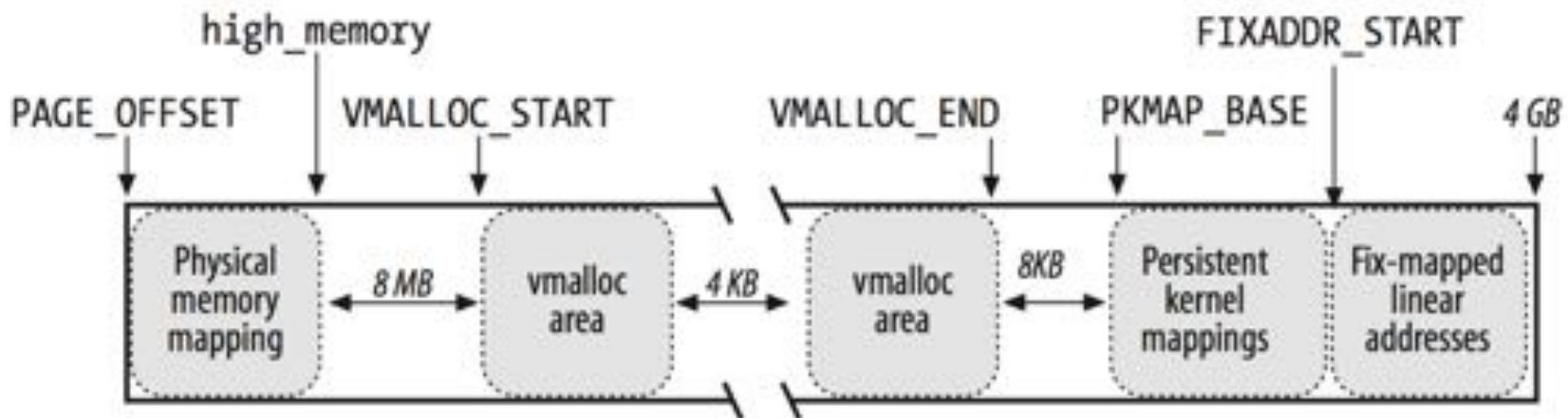
Noncontiguous Memory Area Management

- To avoid external fragmentation
- Descriptor of type *vm_struct* (Table 8-13)
- `get_vm_area()`: look for a free range of linear address
- Allocating a noncontiguous memory area
 - `vmalloc()`
- Releasing a noncontiguous memory area
 - `vfree()`



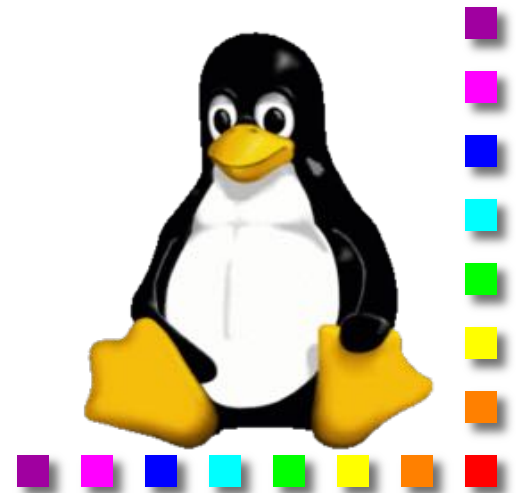
Linear Address Interval Starting from PAGE_OFFSET

- The beginning of the area includes the linear addresses that map the first 896 MB of RAM
- Starting from **PKMAP_BASE** we find the linear addresses used for the persistent kernel mapping of high-memory page frames



Project 3:

Memory Management



上海交通大學

Memory management homework

- Write a module that is called mtest
- When module loaded, module will create a proc fs entry /proc/mtest
- /proc/mtest will accept 3 kind of input
 - "listvma" will print all vma of current process in the format of
start-addr end-addr permission
e.g
0x10000 0x20000 rwx
0x30000 0x40000 r—
 - "findpage addr" will find va->pa translation of address in current process's mm context and print it. If there is not va->pa translation, print "translation not found"
 - "writeval addr val" will change an unsigned long size content in current process's virtual address into val. Note module should write to identity mapping address of addr and verify it from userspace address addr.
- All the print can be done with printk and check result with dmesg.

